Eliminating Timing Channels in Microkernels

Integrating Time Protection Mechanisms into FreeRTOS

Simon V. Brodersen, 09–06–2026

Timing Channels: The Problem

- Timing channels leak information through observable execution time
- Real-world attacks:
  - Cryptographic key extraction (Meltdown, Spectre)
  - Cross-VM information leakage in cloud computing
- The OS scheduler itself is a channel:
  - Priority-based schedulers leak task runnability
  - Shared caches and branch predictors leak state
- **Core challenge:** Closing timing channels at the OS level

Research Question & Contributions

Research Question:

Can time protection mechanisms be integrated into a mainstream OS to mitigate timing channels?

Contributions:

1. A **domain-level scheduler** partitioning execution into fixed time slices
2. **Temporal isolation** via the `fence.t` instruction on domain switches
3. **Evaluation** on QEMU showing constant-time domain scheduling

Why OS-Level Mitigation?

- **User-level mitigations** (e.g., FaCT DSL) guarantee constant-time code
  - Cannot close channels involving OS state — scheduler, interrupts, caches
- **The OS scheduler as a channel:**
  - Priority-based scheduling leaks task runnability
  - Shared microarchitectural state retains cross-task information
- **Ground-up secure OSes** (seL4, S3K) exist, but:
  - Designed from scratch — not applicable to existing systems

Background: Timing Channels

Three categories of timing channels in operating systems:

- **Scheduler-based:** Priority decisions leak runnability info
- **Shared-state:** Caches, TLBs, branch predictors retain cross-task state
- **Delay-based:** Interrupts and locks delay execution predictably

Isolation as the countermeasure:

- *Spatial:* Memory partitioning (MMU / MPU)
- *Temporal:* Ensuring shared resource usage in one domain does not affect another

The `fence.t` Instruction

- Hardware-assisted temporal isolation for RISC-V
- Flushes on-core microarchitectural state on domain switches:
  - L1 caches, TLBs, branch predictors
  - LFSR state, memory arbiters (full flush)
- Overhead in seL4: only **0.7%**
- Software-supported variant (`fence.t.s`) for out-of-order cores: **1.0%**

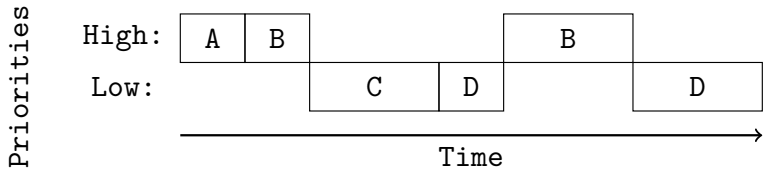
Related Work

- **FaCT (user-level):** DSL for constant-time crypto — insufficient for OS-level channels
- **seL4 Time Protection:** Ground-up secure microkernel
  - OS cloning, cache coloring, deterministic flushing
- **S3K:** Capability-based RTOS with deterministic scheduling
  - Scratchpad memory, constant-time system calls

The Gap: No prior work integrates these mechanisms into an *existing* mainstream OS like FreeRTOS.

Vulnerability: Priority-Based Scheduling

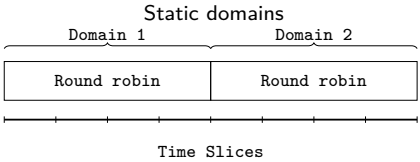
FreeRTOS uses a priority-based preemptive scheduler — inherently unfair and vulnerable.

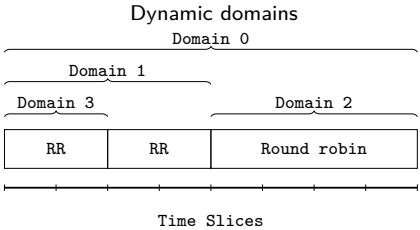


The channel: A high-priority task signals 0 or 1 by sleeping or running, and the low-priority task measures its own wake time.

Design: Domain Scheduling

Chosen approach: Domain-level scheduling





- Domains receive fixed time slices
- Tasks within a domain use standard priority scheduling
- Time protection is enforced only between domains

Implementation: Extending the FreeRTOS API

- Extend `xTaskCreateRestricted` with domain parameters:

```
typedef struct xDOMAIN_PARAMETERS {
  uint32_t ulSliceIndex;   // Start time slice
  uint32_t ulSliceLength;  // Number of slices
} DomainParameters_t;
```

- `NULL` domain params = task stays in caller's domain

Implementation: Domain Switching

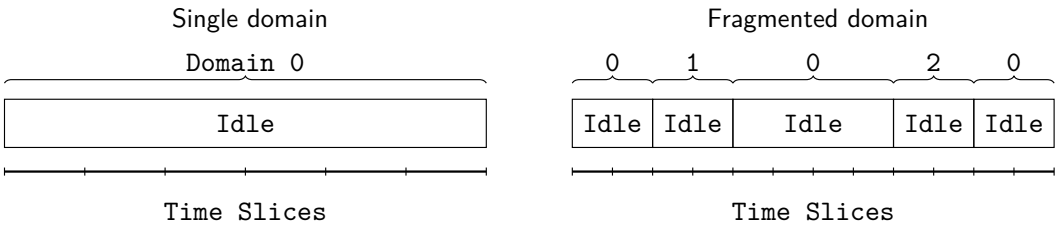
Hook into `xTaskIncrementTick` to check for domain switches:

```
xDomainTick = (xEffectiveTick / NUM_TICKS_PER_SLICE)
                % NUM_TIME_SLICES;

if (xDomainTick >= current + length || xDomainTick < current) {
    temporal_fence_t();          // Flush microarchitectural state
    pxCurrentDomainIndex += length;
    // ... restore next domain's task
}
```

- Flush happens *before* any new-domain data is accessed (temporal_fence_t)
- Prevents cross-domain leakage through shared microarchitectural state

Implementation: Constant-Time Domain Creation



- `xDomains` array indexed by time-slice start
- `uxNext` linked list finds parent blocks in **constant time**
- Index arithmetic splits left / right fragments without scanning

Implementation: Cross-Domain Queues

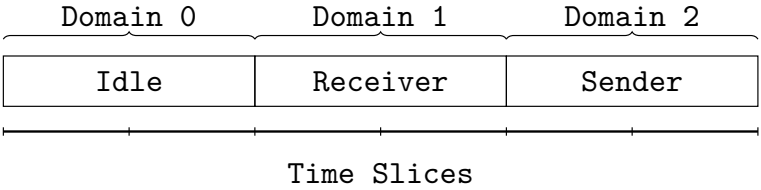
`xQueueSend` moves tasks to the ready list, but by default adds them to the *current* domain's ready list

```
typedef struct tskTaskControlBlock {
    ...
    #if (configENABLE_DOMAINS == 1)
        UBaseType_t uxDomainID;
    #endif
} tskTCB;
```

- `xTaskRemoveFromEventList` now uses the task's `uxDomainID`
- Adds the unblocked task to the *correct* domain's ready list
- Verified with cross-domain `mpu_blinky` demo
- **Caveat:** communicating domains can time each other

Evaluation: Blinky Demo

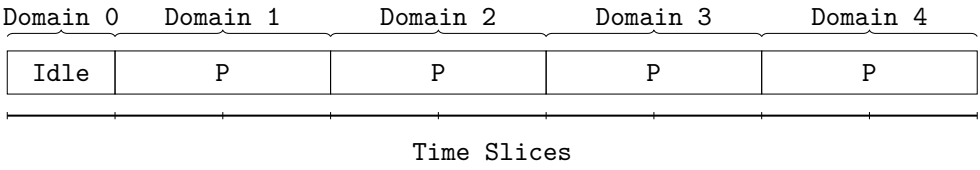
Two tasks in separate domains communicating via a Queue:



- Sender scheduled at domain tick 4, Receiver at domain tick 2
- Round-trip time measured with `rdcycle`: **50,000 cycles ± 1 cycle**
- Demonstrates constant-time domain switching

Evaluation: Multi-Domain Demo

4 domains, each with a privileged task that creates a higher-priority unprivileged task:



- Domain ticks are deterministic (1, 3, 5, 7, ...)
- Priority scheduler still works within each domain
- Trade-off: domain-relative delays, added complexity, reduced throughput

Conclusion

What I showed:

- Time protection mechanisms *can* be integrated into an existing mainstream OS
- Domain-level scheduler achieves constant-time scheduling in QEMU
- Trade-offs
  - **Complexity:**
  - **Performance:**

Conclusion: Full domain isolation is possible and functional in general operating systems.

Future work

- Evaluation on real hardware with `fence.t` support (or gem5)
- Review system calls: critical sections delay scheduling, only user-level protection
- Remaining channels: interrupts, higher-level caches

Selective temporal partitioning might be more feasible in general operating systems.